

AX Ventures Web Application - Project Report

1. Project Overview

This document serves as a comprehensive report on the architecture, technology stack, and scalability roadmap for the AX Ventures web platform. The platform is designed to be a modern, high-performance landing page and application portal for startup founders, featuring dynamic animations, responsive design, and an integrated application pipeline.

2. Architecture

The project strictly follows a modernized **Model-View-Controller (MVC)** architectural pattern adapted for a React + TypeScript environment.

- **Models (`src/models/`):** Defines the data structures, types, and validation logic. E.g., `Application.ts` manages the shape and constraints of the founder application form data.
- **Views (`src/views/`):** Contains all the React UI components, pages, and styling. It is further subdivided into pages (like `AXLandingView.tsx` , `FoundersPage.tsx`) and highly modular UI components (e.g., `MentorsSection.tsx` , `HeroSection.tsx`) within `src/views/components/` . All styling is centralized in `landing.css` .
- **Controllers (`src/controllers/`):** Encapsulates the business logic, state management, and API interactions using custom React hooks. E.g., `useApplicationController.ts` handles the application form state, submission logic, loading states, and error handling, keeping the UI components pure and focused only on rendering.

3. Technology Stack

The platform is built on a robust and modern stack optimized for speed and developer experience:

Frontend

- **Core:** React 19 & TypeScript for type-safe, component-driven UI.
- **Build Tool:** Vite, providing lightning-fast HMR (Hot Module Replacement) and optimized production builds.
- **Routing:** React Router (`react-router-dom`) for seamless SPA (Single Page Application) navigation between the landing page and specialized pages like Events/Founders.
- **Animations:** Framer Motion (`framer-motion`) powers the smooth reveal animations, dynamic hovers, and complex layout transitions.
- **Icons:** Lucide React (`lucide-react`) for clean, customizable vector icons.
- **Styling:** Vanilla CSS (`landing.css`), heavily utilizing CSS Grid, Flexbox, media queries, and CSS variables for theming.

Backend & Analytics

- **Server:** Express.js (`server.js`) acts as the backend to process form submissions.
- **Integrations:**
 - `nodemailer` for routing application submissions to email inboxes.
 - `axios` for HTTP requests from the frontend controller to the backend API.
- **Security:** `helmet` , `express-rate-limit` , and `express-mongo-sanitize` are implemented on the backend to prevent abuse and ensure secure data handling.

- **Analytics:** Integration with `@vercel/analytics` and `@vercel/speed-insights` for real-time performance and traffic monitoring.

4. Scalability Roadmap: Where to Change in the Future

As AX Ventures grows and traffic/feature requests increase, the application will need to scale. Below are the key areas to target for future expansion:

A. Styling Modularity (When adding many more pages)

- **Current State:** A single monolithic `landing.css` file.
- **How to Scale:** Migrate to CSS Modules (e.g., `MentorsSection.module.css`), Tailwind CSS, or Styled Components.
- **Where to Change:** Break apart `src/views/landing.css` and import scoped styles directly into individual components in `src/views/components/`.

B. Global State Management (When adding user authentication/dashboards)

- **Current State:** Local state + custom hook controllers (e.g., `useApplicationController`).
- **How to Scale:** If the app introduces complex user states (e.g., founders logging into a portal), introduce a global state manager like Zustand or Redux Toolkit.
- **Where to Change:** Create a new `src/store/` directory and wrap the application in `src/main.tsx` with the state provider. Update the controllers in `src/controllers/` to dispatch to the global store rather than using local `useState`.

C. Database Integration (When scaling beyond email submissions)

- **Current State:** Applications are sent via email using Nodemailer.
- **How to Scale:** Store applications in a relational database (PostgreSQL) or a BaaS (Supabase/Firebase) so the AX Ventures team can track, filter, and review applications in an admin dashboard.
- **Where to Change:**
 1. Modify `server.js` to connect to a database (e.g., using Prisma ORM).
 2. Update the schema validations in `src/models/Application.ts`.
 3. No changes needed in the Views! The Controller (`useApplicationController.ts`) simply continues to hit the same API endpoint.

D. Serverless Architecture (When traffic spikes)

- **Current State:** A persistent Express server (`server.js`).
- **How to Scale:** To handle sudden traffic spikes without managing server uptime, move the backend logic to Serverless Functions (Vercel Functions or AWS Lambda).
- **Where to Change:** Refactor the routes inside `server.js` into an `api/` folder following the standard serverless deployment structure for your hosting provider (like Vercel).

E. Internationalization (i18n)

- **Current State:** Hardcoded English text.
- **How to Scale:** Use `react-i18next`.
- **Where to Change:** Extract all text strings from `src/views/` components into JSON locale files.